



# Tecnológico de Monterrey

**Nombre del alumno:**

Hania Verónica Rodríguez Cisneros | A00840795

Victor Ivan Sanchez Medina | A01571354

Oscar Tristan Olivares Servín | A00841996

**Nombre de la materia:**

Programación de estructuras de datos y algoritmos fundamentales (Gpo 602)

**Actividad:**

Act 2.3 - Actividad Integral estructura de datos lineales (Evidencia Competencia)

**Docente:**

Carlos Emmanuel Chávez Villarreal

**Fecha de entrega:**

21/04/2026

# Descripción del problema

El problema de esta actividad consiste en procesar un archivo de bitácora que contiene registros de actividad dentro de una red, leer cada línea, almacenarla en una estructura de datos lineal implementada manualmente y después ordenar esa información por dirección IP para poder realizar búsquedas por rango. La restricción importante del ejercicio es que no se pueden usar contenedores de STL, así que tanto la lista doblemente enlazada como la pila deben construirse desde cero.

La idea general del programa es tomar un archivo con muchos registros, convertir cada línea en un objeto que contenga sus datos importantes, almacenar esos objetos dentro de una lista doblemente enlazada y luego ordenar toda la lista por dirección IP. Una vez ordenada, el usuario puede ingresar una IP inicial y una final, y el programa muestra únicamente los registros que caen dentro de ese intervalo. Además, guarda tanto la lista completa ordenada como el resultado de la búsqueda en archivos de texto. En esta versión también se incorpora un análisis de actividad por IP, que permite detectar cuáles direcciones aparecen más veces y cuáles pueden considerarse más recurrentes dentro de la bitácora.

En una idea anterior del algoritmo, el ordenamiento se pensaba usando varias pilas y una lógica más cercana a separar los valores menores, iguales y mayores de manera recursiva. En la implementación actual eso cambia. Ahora se utiliza un *QuickSort iterativo*, en donde la pila no almacena valores ni sublistas completas, sino únicamente los límites de los subrangos de la lista que todavía faltan por ordenar. Eso hace que la solución se parezca más al comportamiento clásico de *QuickSort*, pero adaptado a una lista doblemente enlazada.

# Descripción de las clases y funciones

## Clase LogEntry

La clase **LogEntry** representa una sola entrada de la bitácora. En ella se guardan los componentes de tiempo, la IP convertida a entero, una variable asociada a ataques y el texto de la razón o descripción del evento. Su propósito es encapsular toda la información de un registro para que luego pueda almacenarse, compararse e imprimirse de manera uniforme dentro del programa.

El constructor por defecto inicializa todos los atributos en cero o en cadena vacía. Su función es crear un objeto base, sin información todavía cargada. Como únicamente asigna valores iniciales y no recorre ninguna estructura, su complejidad es  $O(1)$ .

El constructor completo recibe segundos, minutos, hora, día, mes, la IP y la razón del evento. Con esos datos se construye una entrada real del log, lista para ser almacenada en la lista doblemente enlazada. Como solo asigna una cantidad fija de atributos, también tiene complejidad  $O(1)$ .

Los métodos de acceso, como *getSec*, *getMin*, *getHour*, *getDay*, *getMonth*, *getAttacks*, *getReason* y *getIp*, simplemente devuelven el valor de cada atributo del registro. Son funciones directas, por lo que cada una tiene complejidad  $O(1)$ .

La función *compareTime* compara dos registros por fecha y hora. Revisa mes, día, hora, minuto y segundo en ese orden. Aunque en este programa el criterio principal de ordenamiento es la IP, esta función deja preparada la clase para comparaciones cronológicas si se quisieran usar después. Como siempre compara una cantidad fija de campos, su complejidad es  $O(1)$ .

La función *compareIP* compara dos registros usando la IP almacenada como entero. Si la IP de un registro es menor, devuelve un valor negativo; si es mayor, devuelve uno positivo; y si ambas son iguales, devuelve cero. Esta función es la base lógica para los operadores relacionales que más adelante usa el algoritmo de ordenamiento. Su complejidad es  $O(1)$ .

Los operadores de comparación como `==`, `>`, `<`, `>=` y `<=` están sobrecargados para que dos objetos *LogEntry* puedan compararse directamente por IP. Eso vuelve más legible el código del ordenamiento, porque permite trabajar con comparaciones naturales entre registros. Como cada operador depende solamente de una comparación directa entre IPs, todos tienen complejidad  $O(1)$ .

El operador de salida permite imprimir un registro en un formato parecido al de la bitácora original. Reconstruye el nombre del mes, vuelve a descomponer la IP en sus cuatro octetos y muestra toda la información en una sola línea. Como siempre realiza una cantidad fija de operaciones, su complejidad también es  $O(1)$ .

## Clase Node

La clase **Node** representa un nodo de la lista doblemente enlazada. Cada nodo contiene un objeto *LogEntry*, un apuntador al siguiente nodo y otro al nodo anterior. Esta clase es la

base estructural de la lista, ya que gracias a ella los registros pueden enlazarse dinámicamente sin depender de un tamaño fijo.

El constructor del nodo recibe un objeto *LogEntry* y lo guarda dentro del nodo, dejando los apuntadores *next* y *prev* en *nullptr*. Su complejidad es  $O(1)$  porque únicamente realiza asignaciones directas.

El destructor del nodo limpia los apuntadores del nodo actual. En esta implementación no recorre la lista ni libera otros nodos, solamente deja sus referencias en nulo, así que su complejidad es  $O(1)$ .

```
// nodo de la lista doble
class Node {
public:
    // valor guardado en el nodo
    LogEntry value;
    // apuntador al siguiente nodo
    Node* next;
    // apuntador al nodo anterior
    Node* prev;

    // constructor del nodo
    Node(const LogEntry& _value) : value(_value), next(nullptr), prev(nullptr) {}

    // destructor del nodo
    ~Node() {
        next = nullptr;
        prev = nullptr;
    }
};
```

## Clase StackNode

La clase **StackNode** es un nodo auxiliar para la pila. En lugar de guardar un entero o un dato simple, guarda un apuntador a un nodo de la lista doblemente enlazada. Esto es importante porque la pila no almacena registros del archivo como tal, sino referencias a posiciones dentro de la lista, que luego servirán para representar los límites de los subrangos pendientes del *QuickSort*.

El constructor de esta clase recibe un apuntador a un nodo de la lista y lo guarda como contenido del nodo auxiliar, dejando en nulo su apuntador al siguiente elemento de la pila. Como solo realiza asignaciones directas, su complejidad es  $O(1)$ .

## Clase Stack

La clase **Stack** implementa una pila manual usando nodos auxiliares. Su función principal dentro del programa es reemplazar la recursión del *QuickSort*. En lugar de llamar a la misma función una y otra vez, aquí se van guardando en la pila los límites izquierdo y derecho de los subrangos que todavía faltan por ordenar.

El constructor de la pila inicializa la estructura vacía y pone el contador en cero. Como solo fija valores iniciales, su complejidad es  $O(1)$ .

El destructor de la pila elimina todos los elementos que hayan quedado almacenados. Para hacerlo, utiliza repetidamente la operación de extracción hasta vaciarla. Como depende del número de elementos que existan en ese momento, su complejidad es  $O(n)$  respecto al tamaño actual de la pila.

```
// pila que guarda punteros a nodos de la lista doble
class Stack {
private:
    // nodo que queda en la parte superior de la pila
    StackNode* top;
    // cantidad de elementos almacenados
    int count;

public:
    // constructor de la pila
    Stack() : top(nullptr), count(0) {}

    // destructor de la pila
    ~Stack() {
        while (!isEmpty()) {
            pop();
        }
    }
}
```

La función *push* inserta un nuevo elemento en la parte superior de la pila. Esto se hace creando un nuevo nodo auxiliar y ajustando un par de referencias, por lo que su complejidad es  $O(1)$ .

```
// inserta un puntero a nodo en la pila
void push(Node* x) {
    StackNode* temp = new StackNode(x);
    temp->next = top;
    top = temp;
    count++;
}
```

La función *pop* extrae el elemento superior y devuelve el apuntador almacenado. Si la pila está vacía, devuelve *nullptr*. Como solo modifica la cima de la pila, tiene complejidad  $O(1)$ .

La función *peek* devuelve el valor de la cima sin eliminarlo. Solo consulta una referencia, por lo que su complejidad es  $O(1)$ .

La función *isEmpty* indica si la pila está vacía. Como únicamente revisa si la cima es nula, su complejidad es  $O(1)$ .

La función *getSize* devuelve el número actual de elementos almacenados en la pila. Como este valor ya se mantiene actualizado con un contador, su complejidad también es  $O(1)$ .

## Clase DoublyLinkedList

La clase **DoublyLinkedList** es la estructura principal donde se almacenan todos los registros del archivo. Tiene apuntadores a la cabeza y a la cola, además de un contador de tamaño. El uso de una lista doblemente enlazada permite insertar nodos dinámicamente y también recorrer la estructura en ambos sentidos.

El constructor de la lista inicializa la estructura vacía. Como solamente deja en nulo cabeza y cola y pone el tamaño en cero, su complejidad es  $O(1)$ .

El destructor de la lista recorre todos los nodos desde la cabeza hasta el final y los va eliminando uno por uno. Su función es liberar completamente la memoria dinámica utilizada por la lista. Como necesita visitar todos los elementos, su complejidad es  $O(n)$ .

La función *insertFirst* inserta un nuevo nodo al inicio de la lista. Si la lista está vacía, ese mismo nodo pasa a ser tanto la cabeza como la cola. Como solo ajusta unas cuantas referencias, su complejidad es  $O(1)$ .

La función *insertLast* inserta un nodo al final. Si la lista está vacía, utiliza la lógica de inserción al inicio. En condiciones normales solo actualiza la cola y sus enlaces, así que tiene complejidad  $O(1)$ .

La función *insertAt* inserta un nodo en una posición específica. Para poder hacerlo, primero necesita recorrer la lista hasta llegar al índice deseado. Por eso, aunque la inserción final en sí es simple, el costo total depende del recorrido previo y su complejidad es  $O(n)$  en el peor caso.

La función *deleteFirst* elimina el primer nodo de la lista y reajusta la cabeza. Como trabaja únicamente sobre el inicio de la estructura, su complejidad es  $O(1)$ .

La función *deleteLast* elimina el último nodo y actualiza la cola. Mientras no sea una lista vacía o de un solo elemento, la operación sigue siendo directa, así que su complejidad es  $O(1)$ .

La función *deleteAt* elimina el nodo ubicado en una posición concreta. Para llegar a ese nodo primero se requiere recorrer la lista, así que la complejidad total es  $O(n)$  en el peor caso.

La función *find* busca si un valor existe dentro de la lista recorriendo los nodos de manera secuencial. Como podría necesitar revisar todos los elementos, su complejidad es  $O(n)$ .

La función *getAt* obtiene el valor almacenado en una posición dada. Como la lista no tiene acceso directo por índice como un arreglo, necesita avanzar nodo por nodo hasta llegar a la posición buscada, por lo que su complejidad es  $O(n)$ .

La función *updateAt* actualiza el valor de un nodo en cierta posición. Igual que en el caso anterior, primero debe encontrarse el nodo, así que su complejidad es  $O(n)$ .

La función *getSize* devuelve el tamaño actual de la lista. Como ese dato se conserva en un contador, su complejidad es  $O(1)$ .

La función *printForward* imprime la lista desde la cabeza hasta la cola. Como visita todos los nodos una sola vez, su complejidad es  $O(n)$ .

La función *printBackward* imprime la lista desde la cola hasta la cabeza. De igual forma, necesita recorrer todos los nodos, así que su complejidad es  $O(n)$ .

Las funciones *getHead* y *getTail* devuelven la cabeza y la cola de la lista. Son operaciones directas de consulta, por lo que ambas tienen complejidad  $O(1)$ .

## Funciones auxiliares del ordenamiento y procesamiento

La función *swapValues* intercambia los valores de dos nodos sin mover físicamente los nodos dentro de la lista. Esta decisión simplifica bastante el ordenamiento, porque modificar enlaces en una lista doblemente enlazada puede ser más delicado que intercambiar únicamente el contenido. Como solo realiza una cantidad fija de asignaciones, su complejidad es  $O(1)$ .

La función *partition* realiza la partición del *QuickSort* usando una variante del esquema de Lomuto. Toma como pivote el valor del nodo derecho, recorre el rango actual y va dejando a la izquierda los elementos menores o iguales al pivote. Al final coloca el pivote en su posición correcta y devuelve el nodo donde quedó. En términos prácticos, esta función es la que verdaderamente divide el problema en dos partes: lo que debe quedar antes del pivote y lo que debe quedar después. Su complejidad es  $O(m)$ , donde  $m$  representa el número de



nodos del subrango que se está particionando. En el caso más grande, eso equivale a  $O(n)$ .

```
Node* partition(Node* low, Node* high) {
    // se toma como pivote el ultimo nodo del rango actual
    LogEntry pivot = high->value;

    // i marca la frontera entre:
    // 1. los elementos que ya quedaron acomodados a la izquierda
    // 2. los elementos que aun no se han acomodado
    // empieza en nullptr porque todavia no se ha encontrado ningun valor
    // menor o igual al pivote dentro del rango
    Node* i = nullptr;

    // j recorre todo el rango desde low hasta el nodo anterior a high
    // cada vez que encuentra un elemento menor o igual al pivote:
    // 1. avanza i a la siguiente posicion disponible de la parte izquierda
    // 2. intercambia el valor de j con el de i
    // asi, todos los menores o iguales al pivote van quedando al inicio
    for (Node* j = low; j != high; j = j->next) {
        // si el valor actual es menor o igual al pivote, se manda a la izquierda
        if (j->value <= pivot) {
            if (i == nullptr) {
                // el primer elemento menor o igual al pivote debe quedar
                // exactamente al inicio del rango
                i = low;
            } else {
                // los siguientes menores o iguales se colocan despues del anterior
                i = i->next;
            }
            swapValues(i, j);
        }
    }

    // al terminar el recorrido, i debe avanzar una posicion mas:
    // ahi es donde debe colocarse el pivote para que:
    // a la izquierda queden los menores o iguales
    // y a la derecha queden los mayores
    if (i == nullptr) {
        // si nunca hubo elementos menores o iguales, el pivote va al inicio
        i = low;
    } else {
        // si si hubo, el pivote va justo despues del ultimo acomodado
        i = i->next;
    }

    // se coloca el pivote en su sitio correcto
    // despues de este swap, i apunta a la posicion definitiva del pivote
    swapValues(i, high);
    return i;
}
```

La función *quickSortIterative* es la parte central del ordenamiento. Primero toma la cabeza y la cola de la lista como límites iniciales. Después utiliza una pila auxiliar para guardar pares de nodos que representan subrangos pendientes de procesar. Mientras la pila tenga elementos, se saca un rango, se particiona y, si todavía existen elementos a la izquierda o a la derecha del pivote, esos nuevos subrangos se vuelven a guardar en la pila. La diferencia importante con una versión anterior basada en varias pilas es que aquí no se crean grupos físicos de menores y mayores, sino que todo se reacomoda dentro de la misma lista y la pila solo sirve para recordar qué partes faltan por ordenar. Su complejidad temporal es  $O(n \log n)$  en promedio y  $O(n^2)$  en el peor caso. En espacio extra, la pila auxiliar consume  $O(\log n)$  en promedio y  $O(n)$  en el peor caso.

## Funciones auxiliares de conversión y carga

La función *monthToInt* convierte el nombre del mes a un número entero. Esto sirve para transformar el texto leído del archivo en una forma más sencilla de guardar y comparar. Como solo evalúa un conjunto fijo de opciones posibles, su complejidad es  $O(1)$ .

La función *pause* muestra un mensaje para continuar y espera entrada del usuario. En este código no forma parte esencial del flujo principal, pero quedó como apoyo para interacción manual. Del lado del programa su complejidad es  $O(1)$ .

La función *loadLogEntryData* abre el archivo, lee línea por línea, separa cada componente del registro usando un flujo de texto, convierte el mes a entero, transforma la IP a una representación numérica y crea un objeto *LogEntry* que luego se inserta al final de la lista. Si la opción de impresión está activada, también muestra en pantalla cómo se fue separando cada parte del registro. Esta función es la que convierte el archivo de texto original en la estructura de datos real con la que trabaja el programa. Como procesa cada línea una sola vez, su complejidad es  $O(n)$ , donde  $n$  es el número de líneas leídas.

```

void loadLogEntryData(const string& filename, DoublyLinkedList& list, int n = 50, bool print = false) {
    ifstream file(filename);
    string line;
    int i = 0;

    while (getline(file, line) && i < n) {
        stringstream ss(line);
        string s, min, hr, day, month, ip1, ip2, ip3, ip4, port, reason;

        getline(ss, month, ' ');
        getline(ss, day, ' ');
        getline(ss, hr, ':');
        getline(ss, min, ':');
        getline(ss, s, ' ');
        getline(ss, ip1, '.');
        getline(ss, ip2, '.');
        getline(ss, ip3, '.');
        getline(ss, ip4, ' ');
        // getline(ss, port, ' ');
        getline(ss, reason);
        if (print){
            cout << "month: " << month << " " << endl
                << "day: " << day << " " << endl
                << "hour: " << hr << " " << endl
                << "minute: " << min << " " << endl
                << "second: " << s << " " << endl
                << "ip1: " << ip1 << " " << endl
                << "ip2: " << ip2 << " " << endl
                << "ip3: " << ip3 << " " << endl
                << "ip4: " << ip4 << " " << endl
                << "port: " << port << " " << endl
                << "reason: " << reason << endl;
        }

        int _ip = stoi(ip1) * 255 * 255 * 255 + stoi(ip2) * 255 * 255 + stoi(ip3) * 255 + stoi(ip4);
        int _s = stoi(s);
        int _min = stoi(min);
        int _hr = stoi(hr);
        int _day = stoi(day);
        int _month = monthToInt(month);
        // int _port = stoi(port);
        ++i;
        list.insertLast(LogEntry(_s, _min, _hr, _day, _month, _ip, reason));
    }

    file.close();
}

```

La función *ipToInt* recibe una IP en formato de texto, separa sus cuatro octetos y la convierte a entero para que pueda compararse fácilmente dentro del ordenamiento y la búsqueda. Como siempre procesa exactamente cuatro partes, su complejidad es  $O(1)$ .

La función *intToIpString* hace la operación inversa: recibe una IP entera y la vuelve a convertir al formato con puntos para poder imprimirla de forma legible. Al igual que la conversión anterior, su complejidad es  $O(1)$ .

## Funciones de búsqueda, guardado y reporte

La función *printIPRange* recorre toda la lista y muestra únicamente los registros cuya IP cae dentro del intervalo dado por el usuario. Aunque la lista ya está ordenada, esta implementación realiza un recorrido completo y va filtrando con una condición. Es una

solución clara y correcta, aunque todavía podría optimizarse si se aprovechara más directamente el orden de la lista. Su complejidad es  $O(n)$ .

La función *saveIPRangeToFile* realiza prácticamente la misma tarea que la función anterior, pero en vez de imprimir en consola, escribe los registros encontrados dentro del archivo indicado. Como también recorre toda la lista una vez, su complejidad es  $O(n)$ .

La función *saveToFile* recorre toda la lista y guarda todos los registros en un archivo de texto. Se utiliza para exportar la lista ordenada completa. Su complejidad es  $O(n)$ .

La función *countUniqueIPs* cuenta cuántas IP distintas existen dentro de la lista. Como la lista ya está ordenada por IP, basta con recorrerla y agrupar las repeticiones consecutivas. Su complejidad es  $O(n)$ .

```
// cuenta cuantas ips unicas hay recorriendo la lista ya ordenada por ip
int countUniqueIPs(DoublyLinkedList* list) {
    Node* current = list->getHead();

    if (current == nullptr) {
        return 0;
    }

    int uniqueIPs = 0;

    while (current != nullptr) {
        int currentIP = current->value.getIp();
        uniqueIPs++;

        // avanza mientras sigan apareciendo registros de la misma ip
        while (current != nullptr && current->value.getIp() == currentIP) {
            current = current->next;
        }
    }

    return uniqueIPs;
}
```

La función *averageAttemptsPerIP* calcula el promedio de registros por IP usando el total de elementos de la lista entre el número de IP únicas. Como depende de la función anterior para contar esas IPs, su complejidad efectiva es  $O(n)$ .

La función *countIPsOverThreshold* cuenta cuántas IP tienen más de cierto número de apariciones. Aprovecha que los registros iguales ya quedaron juntos después del ordenamiento. Su complejidad es  $O(n)$ .

La función *printTopNIPs* recorre la lista agrupando repeticiones por IP y va manteniendo un top de las direcciones que tienen más intentos. Como el recorrido completo de la lista domina el costo y el ranking manejado es pequeño, su complejidad general es  $O(n)$ .

```
// imprime las top n ips con mas intentos
void printTopNIPs(DoublyLinkedList* list, int n = 5) {
    int topIPs[5] = {0, 0, 0, 0, 0};
    int topCounts[5] = {0, 0, 0, 0, 0};
    Node* current = list->getHead();

    while (current != nullptr) {
        int currentIP = current->value.getIp();
        int attempts = 0;

        // como la lista ya esta ordenada, todas las repeticiones quedan juntas
        while (current != nullptr && current->value.getIp() == currentIP) {
            attempts++;
            current = current->next;
        }

        // inserta la ip en la posicion correcta del top si supera alguno actual
        for (int i = 0; i < n; i++) {
            if (attempts > topCounts[i]) {
                for (int j = n - 1; j > i; j--) {
                    topCounts[j] = topCounts[j - 1];
                    topIPs[j] = topIPs[j - 1];
                }
                topCounts[i] = attempts;
                topIPs[i] = currentIP;
                break;
            }
        }
    }

    cout << "Top " << n << " IPs con mas intentos:" << endl;
    for (int i = 0; i < n; i++) {
        if (topCounts[i] > 0) {
            cout << i + 1 << ". " << intToIpString(topIPs[i])
                << " con " << topCounts[i] << " intentos" << endl;
        }
    }
}
```

La función *printHeavyHitterReport* imprime un resumen general con el número total de registros, la cantidad de IP únicas, el promedio de intentos, el top de IPs y cuántas superan ciertos umbrales. En realidad esta función reúne varias funciones lineales y presenta sus resultados juntos. Por eso, en términos generales, su complejidad sigue siendo  $O(n)$ .

## Descripción del main

La función **main** coordina todo el flujo del programa. Primero define la cantidad máxima de registros que se van a cargar, declara las variables para la IP inicial y final y crea la lista doblemente enlazada donde se almacenarán los logs. Después llama a la función de carga para leer el archivo *bitacora\_noviembre.txt* y guardar los registros dentro de la lista. Una vez

que los datos están cargados, llama a *quickSortIterative* para ordenar toda la lista por dirección IP. Después genera el reporte de *heavy hitters*, que sirve para tener una visión general de qué IPs aparecen más dentro de la bitácora.

A continuación, el programa solicita al usuario una IP inicial y una IP final. Ambas se convierten de texto a entero mediante la función de conversión correspondiente. Si el usuario introduce las IPs al revés, el programa las intercambia para que el rango quede correctamente definido. Después de eso, se muestran en pantalla los registros cuya IP se encuentre dentro de ese intervalo. Finalmente, se guarda la lista completa ordenada en *ordenado.txt* y el resultado de la búsqueda por rango en *busqueda.txt*, y se imprimen mensajes para confirmar que ambos archivos fueron creados.

Tomando en cuenta sus operaciones principales, la función **main** tiene una complejidad total dominada por el ordenamiento. La carga del archivo es  $O(n)$ , el reporte es  $O(n)$ , la búsqueda por rango es  $O(n)$  y el guardado también es  $O(n)$ , pero el *QuickSort* iterativo tiene comportamiento promedio de  $O(n \log n)$  y peor caso de  $O(n^2)$ . Por eso, el comportamiento global del programa queda en  $O(n \log n)$  en promedio y  $O(n^2)$  en el peor caso.

## Qué imprime y qué genera el programa

Durante su ejecución, el programa produce varios tipos de salida. Primero imprime un reporte de *heavy hitters*, donde muestra el total de registros, el número de IP únicas, el promedio de intentos por IP, el top de IPs con más apariciones y cuántas superan ciertos umbrales. Después solicita al usuario dos direcciones IP y, una vez capturadas, imprime en consola todos los registros cuya IP se encuentre dentro de ese rango. Al final muestra dos mensajes de confirmación indicando que se guardó la lista completa ordenada y que se guardó la búsqueda por rango.

Además de la salida en consola, el programa genera dos archivos. El primero es *ordenado.txt*, que contiene toda la lista de registros ya ordenada por IP. El segundo es *busqueda.txt*, que contiene únicamente los registros que cayeron dentro del rango indicado por el usuario.

## Comparación con la versión anterior basada en pilas

En una versión previa de la idea del algoritmo, el ordenamiento se planteaba con varias pilas auxiliares, separando los valores según fueran menores, iguales o mayores al pivote, y resolviendo el problema de forma recursiva. Esa lógica ayudaba a entender conceptualmente cómo dividir el problema, pero también implicaba mover valores entre varias estructuras y reconstruir resultados parciales constantemente. En la implementación actual, el enfoque cambió a un *QuickSort iterativo* más directo. Ya no se crean pilas para almacenar grupos de datos, sino que se utiliza una sola pila auxiliar que guarda rangos de la lista que aún no se han procesado. De esa forma, la partición ocurre dentro de la misma lista y el algoritmo conserva mejor la idea original de *QuickSort*, reduciendo la necesidad de estructuras intermedias y manteniendo control explícito del proceso sin usar recursión.

## Cierre de la sección

En conjunto, el programa resuelve el problema planteado al combinar una lista doblemente enlazada implementada manualmente con un esquema de *QuickSort iterativo* apoyado por una pila propia. Más allá de que el objetivo principal era leer, ordenar, buscar y guardar registros por IP, la solución también permite entender cómo interactúan distintas estructuras lineales en un caso real de procesamiento de bitácoras. El resultado no solo cumple con el requisito funcional de la actividad, sino que también muestra una evolución clara respecto a enfoques anteriores más basados en varias pilas y separaciones más manuales de los datos.

## Conclusiones personales

Hania: A lo largo de esta actividad se logró comprender de mejor manera cómo aplicar estructuras de datos lineales en un problema real, específicamente en el procesamiento de una bitácora. Implementar una lista doblemente enlazada desde cero permitió entender cómo se gestionan los datos de forma dinámica, así como la importancia de recorrerlos y manipularlos correctamente sin el uso de herramientas ya construidas.

Por otro lado, el uso de QuickSort en su versión iterativa ayudó a optimizar el ordenamiento de los registros. Al tener la información organizada por dirección IP, fue posible realizar búsquedas por rango de manera más clara y eficiente, además de facilitar la generación de

reportes que permiten identificar patrones dentro de los datos. Esta actividad me permitió entender cómo distintas estructuras como listas y pilas pueden complementarse para resolver un mismo problema.

Victor:

El QuickSort iterativo funcionó bien para este problema, pero al implementarlo sobre una lista doblemente enlazada se hizo evidente una de sus limitaciones: el esquema de Lomuto siempre elige el último elemento como pivote, lo que lo hace vulnerable a entradas ya ordenadas o casi ordenadas, degradando su complejidad al peor caso de  $O(n^2)$ . Una alternativa más robusta sería Merge Sort, que garantiza  $O(n \log n)$  en todos los casos y se adapta naturalmente a listas enlazadas porque la operación de fusión no requiere acceso aleatorio por índice. La pila auxiliar que implementamos para hacer el QuickSort iterativo también podría reemplazarse por un enfoque más sofisticado usando colas de prioridad o segmentación por rangos de IP desde la carga del archivo, lo que distribuiría el costo del ordenamiento a lo largo del tiempo de lectura en lugar de concentrarlo en una sola pasada. En sistemas de monitoreo de red en producción, donde los registros no dejan de llegar, ese tipo de ordenamiento incremental es mucho más práctico que ordenar todo al final.

La búsqueda por rango de IP, por ejemplo, recorre toda la lista aunque los datos ya estén ordenados. Con un arreglo ordenado, una búsqueda binaria reduciría esa operación de  $O(n)$  a  $O(\log n)$ , lo que en una bitácora con millones de registros representaría una diferencia muy significativa. Otra alternativa, que pudimos ver en clase y empecé a investigar sobre ella, sería usar un árbol binario de búsqueda o un árbol AVL, que se entiende que utiliza estructuras que permiten inserción dinámica y al mismo tiempo búsqueda eficiente en  $O(\log n)$ . En escenarios reales de análisis de tráfico de red, donde los datos llegan en tiempo real y las búsquedas son frecuentes, ese balance entre inserción dinámica y acceso rápido es lo que se necesita.

Oscar: Al analizar la solución desarrollada en esta actividad, considero que el uso de listas enlazadas, ya sean simples o doblemente enlazadas, no necesariamente representa la opción ideal para resolver un problema de ordenamiento y consulta de registros como este, aunque tampoco me parece una mala elección. Más bien, pienso que se trata de una decisión que tiene valor sobre todo desde el punto de vista formativo, porque obliga a comprender con mayor profundidad cómo se almacenan, conectan y recorren los datos dentro de una estructura dinámica.



Desde una perspectiva práctica, siento que el problema de ordenar registros por dirección IP podría resolverse también con otras estructuras de datos lineales, e incluso con herramientas más directas como una base de datos o un arreglo dinámico. En ese sentido, la lista doblemente enlazada no sobresale necesariamente por ser la alternativa más natural para este caso. Sin embargo, eso no le quita mérito a la implementación, porque sí permite trabajar de forma clara con inserciones dinámicas y con el manejo explícito de nodos, algo que ayuda bastante a entender qué está ocurriendo realmente en memoria.

Uno de los aspectos que me pareció más interesantes fue el uso de QuickSort apoyado con una pila para volver iterativo el procedimiento. A primera vista puede sentirse un poco rebuscado, sobre todo porque el problema podría abordarse con soluciones más directas en otros contextos. Aun así, eso no significa que el procedimiento sea ineficiente. Al contrario, me parece valioso porque muestra cómo un algoritmo clásico puede adaptarse a restricciones concretas, como no usar recursión o no depender de estructuras ya implementadas por bibliotecas externas. En ese sentido, la solución tiene un valor importante como ejercicio de lógica y de diseño de estructuras.

También considero importante mencionar una posible debilidad de las listas enlazadas: su dependencia de la integridad de los enlaces entre nodos. Si por algún error un nodo se corrompe o se rompe la referencia hacia el siguiente o el anterior, parte de la información puede quedar inaccesible, aunque siga existiendo en memoria. Esa vulnerabilidad estructural no siempre se nota cuando se piensa solo en el resultado final del programa, pero sí es relevante cuando se analiza la confiabilidad de la estructura. En comparación con otras formas de almacenamiento más contiguas o centralizadas, una lista enlazada depende mucho más de que cada enlace intermedio permanezca correcto.

A pesar de eso, también reconozco que trabajar con nodos que almacenan objetos de clase resulta cómodo y expresivo para organizar este tipo de información. Cada nodo encapsula un registro completo, y eso hace que la estructura tenga cierto orden conceptual: no se están moviendo datos sueltos, sino entidades con sentido. Desde esa perspectiva, la aplicación de nodos sí me parece adecuada para representar los registros de la bitácora y manipularlos durante el ordenamiento.

En conclusión, mi postura es que la lista doblemente enlazada fue una estructura útil para fines académicos y para comprender mejor la relación entre nodos, memoria, ordenamiento y recorrido de datos, pero no diría que sea la opción más natural o más robusta para un problema como este en un entorno práctico. Aun así, la implementación demuestra que puede funcionar correctamente y que, combinada con un algoritmo como QuickSort iterativo con pila, ofrece una solución interesante, eficiente y conceptualmente rica para analizar.